SQE Assignment 1 – what was done and why

Study summary for TestLibrary.java, using the terms from the lecture and practical session slides.

1. Results

Check	Result
mvn clean test	163 tests, 0 failures (all pass against the provided Library.java)
Course checker (assignment1.jar)	Grade 100.0, passed 8 out of 8 mutations. Ran on Java 21, the jar asks for 17.
Statement coverage	122/122 lines of Library (100%)
Mutation testing (PIT tool, my own run)	367 mutants: 366 killed, 1 equivalent

Submit only **TestLibrary.java**. Do not submit the TestLibrary.grade file the checker creates next to it.

2. Basic terms (Practical Session 1)

Term from the slides	Where it is in our file
Software Under Test (SUT)	The Library class. Created fresh in setUp().
Test Data	The inputs: ISBN strings, user ids, author names.
Expected Output / Oracle	What we assert: the exception type and message, or the call that should reach the database.
Sunny day (valid, typical inputs)	One success test per method, e.g. givenValidNewBook_whenAddBook_...
Rainy day (invalid, boundary and extreme inputs)	One test per failing check: null, empty, wrong length, wrong check digit...
Boundary conditions	ISBN with 12 / 14 digits, user id with 11 / 13 digits, author with one letter, notification that works on exactly the 5th attempt.

Four parts to a test (AAA Pattern)

Part	In our tests
1. Setup (**Arrange**)	@BeforeEach setUp() + the "// Arrange" block: instantiate, then stubbing.
2. Invocation (**Act**)	One call to one Library method.
3. Assessment (**Assert**)	Verify the interactions, then assert outputs.
4. **Teardown**	@AfterEach tearDown() gives System.out / System.err back.

Testing convention – test method name

Format from the slides: **Given<Condition>_When<Method>_Then<Result>**. The TA wrote in the forum that both versions are accepted and that he recommends the lowercase one, so the file uses **givenBorrowedBook_whenReturnBook_thenBookReturnedAndSavedInDatabase**.

JUnit 5 annotations and asserts used

Annotation / assert	Use in the file
@BeforeEach	New mocks and a new Library so each test starts clean.

@AfterEach	Restore the console streams.
@Test	A single scenario.
@ParameterizedTest + @ValueSource	Same test over many literal values (all the invalid ISBNs, authors, ids).
@NullAndEmptySource	Adds null and "" to a parameterized test.
assertThrows	Checks the exception type; we then assertEquals on its message.
assertEquals / assertTrue / assertFalse / assertSame	Expected vs. actual result.

Do & Do-not: simple tests with a single aspect in each test, no random parameters (all values are fixed constants), edge cases and regular cases, fast tests.

3. Isolation (Practical Session 2)

"Unit tests must operate independently, free from external dependencies." So the Library never talks to a real class:

Dependency	Test double
DatabaseService, ReviewService, NotificationService	@Mock (interfaces with no implementation – exactly the "when to use mocks" case)
Book, User	@Mock too ("avoid use of external classes"). A bug in Book cannot fail a Library test.

- **Mockito set up:** MockitoAnnotations.openMocks(this) inside @BeforeEach. The other option on the slide, @ExtendWith(MockitoExtension.class), is not possible because the pom does not include that library.
- **Dependency Injection / constructor injection:** the mocks enter through new Library(databaseService, reviewService). This is "Designing for Testability".
- **Spy:** not used. A spy wraps a real object, and the TA wrote that Library is the SUT and we do not spy on it. getBookByISBN calls notifyUserWithBookReviews internally, so we just arrange the mocks of the services that both need.

Test doubles (Lecture 03)

Double	Definition on the slide	Our code
Stub	control indirect input	when(databaseService.getBookByISBN(..)).thenReturn(book) when(reviewService.getReviewsForBook(..)).thenThrow(new ReviewException(..))
Mock	adds expectations	verify(databaseService).addBook(isbn, book) verify(user, times(5)).sendNotification(..) verify(book, never()).borrow()
Spy	allows retro inspection	not used
Dummy / Fake	never used / simplified impl.	not needed

In Mockito both stubs and mocks are called "mock" – the same @Mock object is a stub when we use when(), and a mock when we use verify().

State verification vs. behavior verification: most Library methods are void, so there is no return value to assert. We use behavior verification (verify) to check what the Library did to its dependencies. Only getBookByISBN returns something, and there we use assertSame.

4. Rules from the course forum (TA answers)

Rule	How the file follows it
All tests must pass against the provided implementation	163/163 pass. We test the behavior of the code as it is (e.g. a title of one space is accepted).
One reason for failure in each test	setUp() makes a fully valid book and user; each test breaks exactly one thing. The tests that mixed two invalid inputs were removed.
Check no side effects after an exception	verifyNoInteractions(databaseService), verify(.., never()).borrowBook(..) and so on in every failing scenario.
Private methods are tested indirectly	isISBNValid and isAuthorValid are tested through addBook.
Library is the SUT, do not spy on it	Only the external dependencies are mocked.
English only in names and strings	The file is pure ASCII. When I tried non-English letters the checker printed an error, so they were removed.
Two different special chars in a row are valid (A.-B)	"A.-B" and "Robert.-Martin" are in the valid authors list.
Order of checks	If a mutant moves the database access before the input validation, the test gets a different exception and fails – which kills the mutant. We get this without mixing invalid inputs because an un-stubbed mock returns null.

5. Coverage and mutation testing (Lectures 03 and 04)

- **Statement coverage** (every statement is executed by at least one test case): 100%.
- **Branch coverage** (every outgoing edge of a node): each "if" in Library has a test for true and a test for false. This is why there is one test per check, in the same order as the code.
- **Mutation**: a small modification of the code; the modified code is a **mutant** (example from the slide: x > 1 becomes x >= 1).
- **Killed**: a test failed on the mutant. **Equivalent**: behaves like the original. **Stubborn**: the test set is insufficient.
- **Mutation score** = 100 * D / (N - E). Ours: D = 366, N = 367, E = 1, so 100.
- The one equivalent mutant: the loop in isAuthorValid starts at index 1 instead of 0. Index 0 was already checked to be a letter, so nothing changes.

Things added only to kill mutants

What the test checks	Mutant it kills
The exact exception message, not only the type	Two checks swapped or a wrong message thrown.
verify(.., never()) after an exception	The code continues and saves anyway.
Exact arguments: borrowBook(ISBN, userId)	Arguments swapped or changed.
times(5) on sendNotification, and success on the 5th attempt	retryCount < 5 changed to <= 5 or < 4.
reviewService.close() called once, also on failure	The finally block removed.
InOrder: book.borrow() before databaseService.borrowBook()	Order of the operations changed.
ISBN whose check digit is 0 (9783161484100)	The "if (checkDigit == 10)" line removed.
ISBN with two digits switched (9870306406157)	The x1 / x3 weights changed.
What is printed to System.out / System.err	A print line removed.

6. Edge cases that were tried

- **ISBN:** null, empty, 12 and 14 digits, letters, wrong check digit, spaces before / after, new line at the end, only hyphens, plus sign, all zeros (valid), hyphens at the start / end / between every digit (valid).
- **User id:** null, empty, 11 / 13 / 23 digits, letters, spaces, sign, dot, new line at the end, all zeros and all nines (valid).
- **Author:** starts or ends with a non-letter, digit or bad char in the middle, tab, new line, comma, the four consecutive pairs (two hyphens, two spaces, two dots, two apostrophes), one letter (valid), different special chars in a row (valid).
- **Reviews:** null list, empty list, ReviewException, another kind of exception (not replaced, service still closed).
- **Notification:** works first time, fails 2 times, fails 4 times, fails always, another kind of exception (no retry).
- **getBookByISBN:** returns the book even when the notification part fails in any of the ways above.

7. Before you submit

- The forum says submission is **in pairs only**.
- The TA asked everyone to re-download Library.java and assignment1.jar from the updated zip. Make sure your copy is the new one and run the checker again on it.
- If you can, run the checker once on Java 17 (the official version).